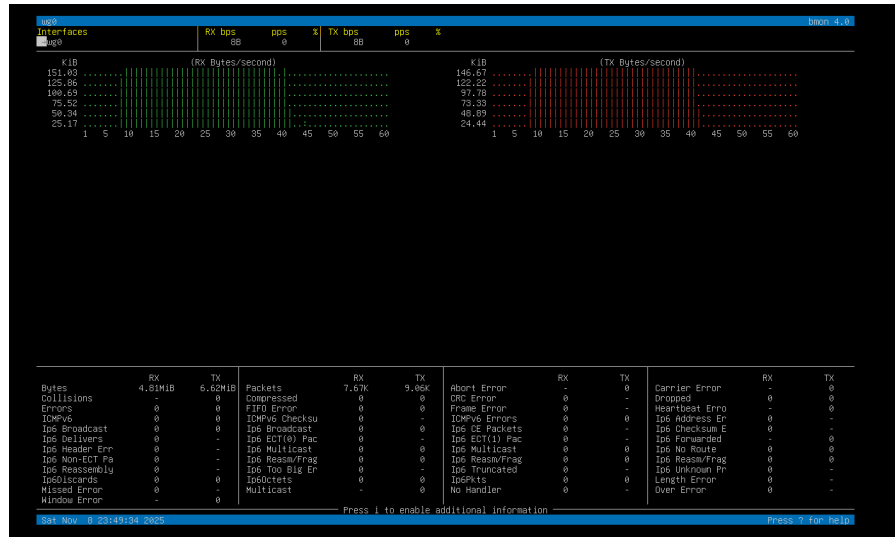


CyberVPN: Building my own secure VPN with WireGuard



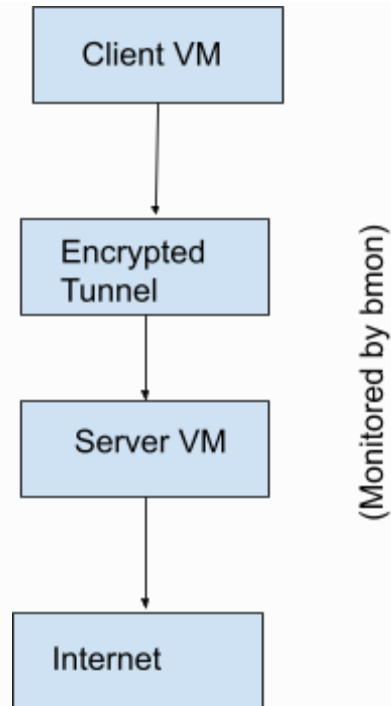
Screenshot of bmon capturing traffic spikes

Introduction

I made a VPN for this project. A VPN, or Virtual Private Network, is a secure way to connect to the internet or another network, keeping your data private by creating a tunnel that hides it from others. It's used to protect privacy, access blocked sites, or safely work remotely.

Overview

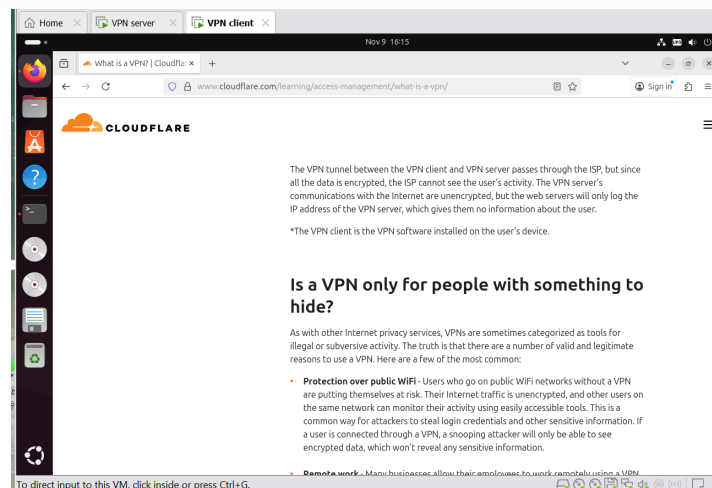
The VPN works by setting up a secure tunnel between two virtual machines (VMs)—one as the server and one as the client—using WireGuard software. The server acts like a gateway, routing the client's internet traffic through this encrypted channel. To show it's working, we'll use bmon, a tool that displays real-time traffic graphs on the server's VPN interface (wg0). Imagine a tunnel like this:



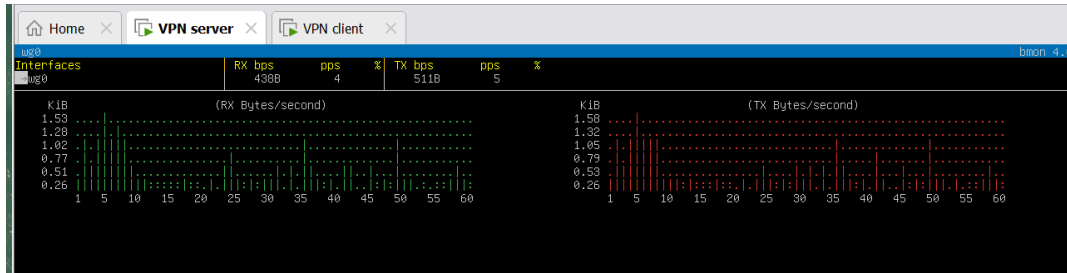
Flow diagram

Demo:

On my client machine i searched on Google



Then whatever i browsed all the traffic went through the **wg0** interface of the VPN server



Motivation

I wanted to learn how a VPN works, so what better way to learn than to build one? Hands-on experience helps me understand the tech behind it.

Methodologies

Prerequisites

- Ubuntu server ISO
- Ubuntu Desktop ISO
- VMware workstation pro
- Wireguard installed on both VMs
- Install bmon on server
- Set up your VMware network to NAT for both VMs

VPN Server Setup

This details the step-by-step process of setting up a WireGuard VPN server on an Ubuntu virtual machine (VM) using VMware Workstation Player. Each command is explained in simple English so anyone can understand why it was run and what it did. Screenshots and outputs of each command are included for clarity.

Step 1: Install WireGuard

Command:

sudo apt update # Ran to refresh software list.

sudo apt install wireguard -y # Ran to install WireGuard, auto confirming prompts.

```

cc min avg max mdev = 28.613/84.130/138.334/36.192 ms
vpnadmin@wg-server-vpn:~$ sudo apt install wireguard -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  wireguard-tools
The following NEW packages will be installed:
  wireguard wireguard-tools
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 92.2 kB of archives.
After this operation, 345 kB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 wireguard-tools amd64 1.0.20210914-1ubuntu4 [89.1 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 wireguard all 1.0.20210914-1ubuntu4 [3,086 B]
Fetched 92.2 kB in 1s (157 kB/s)
Selecting previously unselected package wireguard-tools.

```

What happened?

- Installed WireGuard with ~10 dependencies.
- VM is now ready to act as a VPN server.

Step 2: Generate Keys

Commands:

sudo mkdir -p /etc/wireguard # Created /etc/wireguard folder for VPN keys.

*wg genkey | sudo tee /etc/wireguard/private.key | wg pubkey | sudo tee /etc/wireguard/public.key
Generated private and public key pair. Saved keys in /etc/wireguard.*

sudo chmod 600 /etc/wireguard/private.key # Set private key permissions to admin-only (chmod 600).

```

vpnadmin@wg-server-vpn:~$ sudo mkdir -p /etc/wireguard
vpnadmin@wg-server-vpn:~$ wg genkey | sudo tee /etc/wireguard/private.key | wg pubkey | sudo tee /etc/wireguard/public.
vpnadmin@wg-server-vpn:~$

```

What happened?

- Private key secured; public key ready for client use.

Step 3: Create Server Config

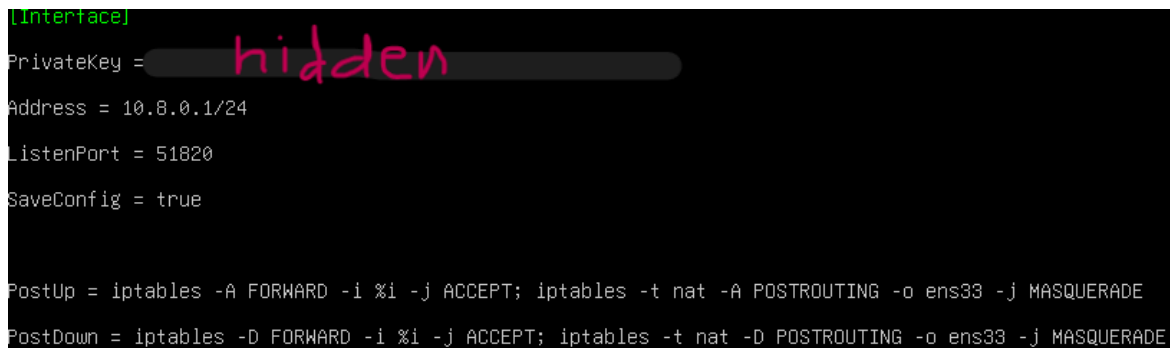
Command:

sudo nano /etc/wireguard/wg0.conf # Opened file to create VPN config.

Wg0.conf:

```
[Interface]
PrivateKey = <redacted for security>
Address = 10.8.0.1/24
ListenPort = 51820
SaveConfig = true

PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -t nat -A POSTROUTING -o ens33 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -t nat -D POSTROUTING -o ens33 -j MASQUERADE
```



```
[Interface]
PrivateKey = hidden
Address = 10.8.0.1/24
ListenPort = 51820
SaveConfig = true

PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -t nat -A POSTROUTING -o ens33 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -t nat -D POSTROUTING -o ens33 -j MASQUERADE
```

What happened?

- Added private key, VPN IP range (10.8.0.1/24), and port (51820).
- Enabled SaveConfig = true for auto-saving settings.
- Added PostUp/PostDown rules for NAT and traffic forwarding via ens33.
- Configured server to handle VPN connections and routing.

Step 4: Enable IP Forwarding

Commands:

sudo nano /etc/sysctl.conf # Edited /etc/sysctl.conf to enable IP forwarding.

Add in the file: net.ipv4.ip_forward=1 # to allow traffic routing.



```
net.ipv4.ip_forward=1
```

sudo sysctl -p

```
vpnadmin@wg-server-vpn:~$ sudo sysctl -p
net.ipv4.ip_forward = 1
```

What happened?

- Enabled server to share internet with client via VPN.

Step 5: Start WireGuard

Commands:

sudo wg-quick up wg0 # Started VPN server

sudo systemctl enable wg-quick@wg0 # Enabled auto-start on boot

sudo ufw allow 51820/udp # If UFW is active # Opened UDP port 51820 in firewall

```
vpnadmin@wg-server-vpn:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.8.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o ens33 -j MASQUERADE
vpnadmin@wg-server-vpn:~$ sudo systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /usr/lib/systemd/system/wg-quick@.service
vpnadmin@wg-server-vpn:~$ sudo ufw allow 51820/udp
Rules updated
Rules updated (v6)
vpnadmin@wg-server-vpn:~$
```

Check:

sudo wg show # Verified VPN status

```
vpnadmin@wg-server-vpn:~$ sudo wg show
interface: wg0
  public key: <hidden>
  private key: (hidden)
  listening port: 51820
vpnadmin@wg-server-vpn:~$ _
```

What Happened?

- VPN server is active and ready for connections.

Now what ?

- WireGuard VPN server successfully set up on Ubuntu VM.
- Server running and listening on port 51820.
- IP forwarding enabled for traffic routing.
- Firewall configured to allow VPN traffic.
- `sudo wg show` confirmed wg0 interface is active.

VPN client setup

This outlines the step-by-step process of setting up a WireGuard VPN client on an Ubuntu virtual machine (VM) using VMware Workstation Player. Each command is explained in simple English so anyone can understand why it was run and what it did. Screenshots and outputs of each command are included for clarity.

Step 1: Install WireGuard

Command:

sudo apt update # Ran to refresh software list.

sudo apt install wireguard -y # Ran to install WireGuard, auto confirming prompts.

```
vpnclient@vpnclient-VMware-Virtual-Platform:~$ sudo apt update
[sudo] password for vpnclient:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
127 packages can be upgraded. Run 'apt list --upgradable' to see them.
vpnclient@vpnclient-VMware-Virtual-Platform:~$ sudo apt install wireguard -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

What happened?

- Installed WireGuard with ~10 dependencies.
- Client VM ready to connect to VPN.

Step 2: Generate Keys

Commands:

sudo mkdir -p /etc/wireguard # Created /etc/wireguard folder for VPN keys.

wg genkey | sudo tee /etc/wireguard/private.key | wg pubkey | sudo tee /etc/wireguard/public.key
Generated private and public key pair. Saved keys in /etc/wireguard.

sudo chmod 600 /etc/wireguard/private.key # Set private key permissions to admin-only (chmod 600).

```
vpnclient@vpnclient-VMware-Virtual-Platform:~$ wg genkey | sudo tee /etc/wireguard/private.key | wg pubkey | sudo tee /etc/wireguard/public.key
```

What happened?

- Private key secured; public key ready for client use.

Step 3: Create Client Config

Command:

sudo nano /etc/wireguard/wg0.conf # created and opened file

```
vpnclient@vpnclient-VMware-Virtual-Platform:~$ sudo nano /etc/wireguard/wg0.conf
```

Wg0.conf:

```
[Interface]
PrivateKey = <redacted for security>
Address = 10.8.0.2/24
DNS = 8.8.8.8 # Google DNS for client

[Peer]
PublicKey = <redacted for security>
AllowedIPs = 0.0.0.0/0 # Routes all traffic through VPN
Endpoint = <SERVER_VM_IP>:51820
PersistentKeepalive = 25
```

GNU nano 7.2 /etc/wireguard/wg0.conf *

```
[Interface]
PrivateKey = <client private key>
Address = 10.8.0.2/24
DNS = 8.8.8.8

[Peer]
PublicKey = <server public key>
AllowedIPs = 0.0.0.0/0
Endpoint = <Server vm ip>:51820
PersistentKeepalive = 25
```

What happened?

- Added private key, VPN IP (10.8.0.2/24), and DNS (8.8.8.8).
- Added server's public key and endpoint (<SERVER_VM_IP>:51820).
- Set AllowedIPs = 0.0.0.0/0 to route all traffic through VPN.
- Added PersistentKeepalive = 25 to maintain connection.
- Client config ready to connect to VPN server.

Step 4: Add Client to Server

Command:

sudo wg set wg0 peer <CLIENT_PUBLIC_KEY> allowed-ips 10.8.0.2/32

```
vpnadmin@wg-server-vpn:~$ sudo wg set wg0 peer <CLIENT_PUBLIC_KEY> allowed-ips 10.8.0.2/32
vpnadmin@wg-server-vpn:~$
```

What happened?

- Added client's public key to server configuration.
- Assigned client VPN IP 10.8.0.2.
- Set AllowedIPs = 10.8.0.2/32 for secure traffic restriction.
- Client authorized to connect to VPN server.

Step 5: Connect Client

Commands:

sudo wg-quick up wg0

Test:

ping 10.8.0.2

```
vpnclient@vpnclient-VMware-Virtual-Platform:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.8.0.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] resolvconf -a wg0 -m 0 -x
[#] wg set wg0 fwmark 51820
[#] ip -4 rule add not fwmark 51820 table 51820
[#] ip -4 rule add table main suppress_prefixlength 0
[#] ip -4 route add 0.0.0.0/0 dev wg0 table 51820
[#] sysctl -q net.ipv4.conf.all.src_valid_mark=1
[#] nft -f /dev/fd/63
vpnclient@vpnclient-VMware-Virtual-Platform:~$ ping 10.8.0.2
PING 10.8.0.2 (10.8.0.2) 56(84) bytes of data:
64 bytes from 10.8.0.2: icmp_seq=1 ttl=64 time=0.051 ms
64 bytes from 10.8.0.2: icmp_seq=2 ttl=64 time=0.076 ms
64 bytes from 10.8.0.2: icmp_seq=3 ttl=64 time=0.098 ms
64 bytes from 10.8.0.2: icmp_seq=4 ttl=64 time=0.075 ms
64 bytes from 10.8.0.2: icmp_seq=5 ttl=64 time=0.075 ms
64 bytes from 10.8.0.2: icmp_seq=6 ttl=64 time=0.093 ms
64 bytes from 10.8.0.2: icmp_seq=7 ttl=64 time=0.099 ms
64 bytes from 10.8.0.2: icmp_seq=8 ttl=64 time=0.102 ms
^C
--- 10.8.0.2 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7172ms
rtt min/avg/max/mdev = 0.051/0.083/0.102/0.016 ms
```

On Server:

sudo wg show

```

vpnadmin@wg-server-vpn:~$ sudo wg show
[sudo] password for vpnadmin:
interface: wg0
  public key: [REDACTED]
  private key: (hidden)
  listening port: 51820

peer: [REDACTED]
  endpoint: [REDACTED]
  allowed ips: [REDACTED]
  latest handshake: 1 minute, 42 seconds ago
  transfer: 1.13 KiB received, 1.10 KiB sent
vpnadmin@wg-server-vpn:~$

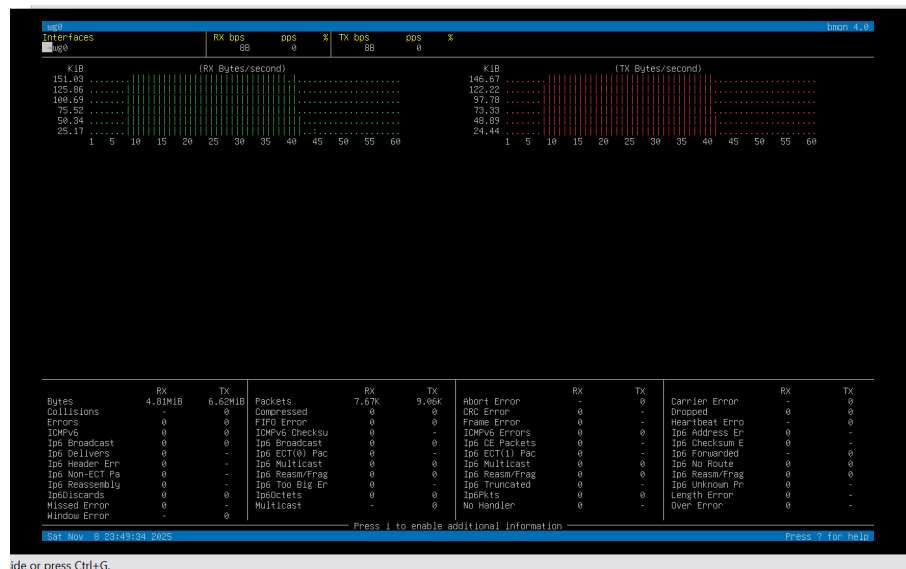
```

What happened?

- Started VPN on client
- Tested connection to server using ping 10.8.0.1.
- Checked handshake on server using `sudo wg show`.
- Confirmed VPN connection between client and server.

Monitor with bmon on Server

- On the server VM, in a new terminal, run:
`sudo bmon -p wg0`



Simulated VPN Traffic Spike with Ping on bmon.

What happened?

- **bmon** (Bandwidth Monitor) shows real-time network bandwidth usage per interface.
- Monitored wg0 to observe VPN traffic flow.

- **RX** = data received, **TX** = data transmitted.
- Expected small RX/TX spikes (~10–50 kB/s) from ping traffic.
- Watch for bar/number spikes showing activity.

Now what ?

- WireGuard client successfully connected to server.
- ping 10.8.0.1 confirmed reachability to server's VPN IP.
- sudo wg show displayed handshake = connection verified.
- VPN fully functional for secure communication between client and server.

End Result

After following these steps, my VPN is up and running! The server and client connect through the encrypted tunnel, and bmon shows traffic spikes, proving it works. I can disconnect with sudo wg-quick down wg0 when done.

What I Have Learned

I learned how tunneling works, how to set up a VPN server, and how to configure both client and server. It was a hands-on way to grasp networking and security.

Conclusion

Building this VPN was a cool challenge that paid off. I now get how VPNs keep data safe and can set one up myself.